SSGAC/CESR Social-Science Genetics Methods Workshop

# Statistical genetics on All of Us

## Lab 1: a phenotype, a polygenic index, and a regression on v9 data

Jeffrey M. Swigert, PhD

Center for Economic and Social Research, University of Southern California

Fall 2026

Generously supported by NIH/NIA Grant R24-AG065184 and the All of Us Research Program

# Do this now: start your cloud environment

In the workshop workspace: **Apps** tab → **+ New app instance** → **JupyterLab**. Creation takes a few minutes; we continue while it boots.

| Setting | Choose | Why |
|---|---|---|
| Environment | Standard VM | The Spark cluster option is for Hail. |
| Machine | `n1-standard-8`: 8 CPUs, 30 GB memory | The first machine in the menu. Enough for one chromosome. |
| Disk | 100 GB | Today uses under 5 GB. Disks grow, never shrink. |
| GPU | None | Nothing today uses a GPU. |
| Autostop | On, 1 hour | Stops the VM after 1 idle hour: a clean stop when the lab is over. |

About $0.41 per hour while running; a stopped VM bills only for its disk.

# What we will do today

**Lab 1**, or "this lab," means this one-hour session. Seven steps, each one script:

| Step | What we do | Script (in `scripts/`) |
|---|---|---|
| 0 | Get the code, fill in settings | `00_setup.sh` |
| 1 | Put chromosome-22 genotypes and ancestry data on the VM | `01_fetch_genotypes.sh` |
| 2 | Build a height phenotype from the CDR | `02_build_phenotype.sh` |
| 3 | Summarize it: a summary table and two figures | `03_explore_phenotype.sh` |
| 4 | Build a polygenic index (PGI); check its histogram | `04_build_pgi.sh` |
| 5 | Regress height on the PGI and covariates | `05_pgi_regression.sh` |
| 6 | Save the run; download aggregate results | `06_save_run.sh` |

For each step: a slide on what it does, then a **Run it** slide with the commands. We run each step together before moving on.

# The repository has the details

`github.com/jeffreyswigert/aou-statgen-lab`

**Open it in a browser tab now and keep it open.** The slides are brief on purpose; the repository explains every step in more detail:

- **README**: what each step does, why, and where the data and weights come from.
- **The scripts**: every line has a comment saying what it does, why, and what it looks like in your own project.
- `COMMANDS.txt`: every command from today, in order, with a note on each. Copy and paste from it.
- `cheatsheet.pdf` and `slides.pdf`: for after class.

If a step goes by too fast today, its script and the README are where the explanation lives.

# All of Us data use rules

| Rule | In this lab |
|---|---|
| Participant-level data stays in the Workbench: no copying, downloading, or screenshots. | Person-level files live in `lab_data/` and `work/` only. |
| No count of 1–20 participants may be shared, directly or by calculation. | Step 6 checks `results/` before anything is saved or downloaded. |
| Downloads are monitored; downloading participant-level data triggers an alert. | Downloads come from `results/` only. |
| No participant data or counts under 20 in code on GitHub. | `.gitignore` excludes data and output folders. |
| Notify the program 2 weeks before a publication or presentation. | For your own projects. |

# GitHub: what it is and how we use it

**Git** records versions of the files in a folder; each saved version is a *commit*. **GitHub** hosts a copy of the folder (a *repository*) online.

Today it does three things:

1. **Gets the code onto the VM**: `git clone` once, `git pull` for updates.
2. **Records which code produced a result**: step 6 writes the commit ID into `results/run_manifest.txt`.
3. **Runs identical code** in every participant's workspace.

GitHub is outside the Workbench. A repository holds code, settings templates, and documentation. It should never hold data, results, filled-in settings, or notebook outputs.

# `.gitignore`: keeping data out of GitHub

A `.gitignore` file lists files and folders that git never tracks. This repository's:

```
config.sh              # your filled-in settings
lab_data/              # copied-in data (person-level)
work/                  # intermediate files (person-level)
results/               # aggregate outputs (reviewed and downloaded by hand)
runs/                  # saved archives
.ipynb_checkpoints/    # notebook autosaves
```

**For any project:** ignore data and output folders by name; commit a settings template and ignore the filled-in copy; clear notebook outputs before committing.

More practices in the README, "GitHub in this workflow."

# Run it – Step 0: get the code and fill in settings

JupyterLab Launcher → **Terminal**. One command at a time (or paste from `COMMANDS.txt`):

```
git clone https://github.com/jeffreyswigert/aou-statgen-lab.git   # copy the repository here
cd aou-statgen-lab                  # move into it
cp config.example.sh config.sh      # your own copy of the settings
nano config.sh                      # fill in the two values below, then save and exit
bash scripts/00_setup.sh            # check tools, settings, disk
```

The two values (shown on screen; also on the workspace's page: Resources tab → bucket; Overview → Google project):

```
export WORKSHOP_BUCKET="gs://..."    # the workshop workspace's bucket
export BILLING_PROJECT="..."         # the Google project billed for queries and copies
```

**Save and exit nano:** `Ctrl-O`, `Enter`, `Ctrl-X`. If that does not work in your browser (Windows): `Ctrl-X`, `Y`, `Enter`.

**Expected:** the last line reads `Setup check passed.`

# Where today's data are

1. **The CDR**, in BigQuery: `wb-silky-artichoke-2408.C2025Q4R6`

   Tables `measurement`, `person`, `concept` → the height phenotype (step 2)

2. **The All of Us controlled bucket**: `gs://vwb-aou-datasets-controlled/v9/`

   `wgs/short_read/snpindel/aux/ancestry/ancestry_preds.tsv`: predicted ancestry and 16 PCs per person (step 1)

   `wgs/short_read/snpindel/acaf_threshold/`: whole-genome genotypes, one file set per chromosome (next slide)

3. **The workshop bucket**: `$WORKSHOP_BUCKET`

   `genotypes/chr22_hm3.{bed,bim,fam}`: the chromosome-22 subset, ready made (step 1)

   `pgi_workshop/height_weights.txt`: height PGI weights, Yengo et al. (2022) (step 4)

4. **The repository**: `data/hm3_chr22_hg38.tsv`, the public HapMap3 variant list

The controlled bucket is **requester-pays**: each copy names who pays (`-billing-project`).

# Where the genotypes come from

- All of Us publishes whole-genome genotypes (the ACAF callset) per chromosome: for chromosome 22, 25 GB (PLINK 2) or 276 GB (PLINK 1). No HapMap3 subset.
- **HapMap3 variants**: about 1.2 million common variants that most GWAS report; the standard set for PGIs.
- So a PGI project starts by keeping those variants and renaming them to rsIDs (`scripts/subset_aou_genotypes.sh`):

```
# keep the HapMap3 variants and the chosen people; name variants chr22:pos:ref:alt
plink2 --pfile acaf --set-all-var-ids 'chr@:#:$r:$a' \
       --extract extract.txt --keep keep.txt --make-bed --out hm3_aouids
# rename the variants to rsIDs, to match the PGI weights
plink2 --bfile hm3_aouids --update-name rename.txt --make-bed --out chr22_hm3
```

I ran this before class; step 1 copies the result. (Building it yourself: "On your own" slides, and the README.)

# Run it – Step 1: the genotypes and the ancestry file

```
bash scripts/01_fetch_genotypes.sh      # under a minute; start it, then read on
```

What the script does, line by line (`scripts/01_fetch_genotypes.sh`):

| Line | What it does | In your own project |
|---|---|---|
| `source scripts/common.sh` | loads config.sh, the safety switches, the billing flag | one shared setup file |
| `mkdir -p lab_data` | a folder for copied-in data | data outside the code folder |
| `for ext in bed bim fam; do ... done` | runs the copy three times, once per extension | files that travel together |
| `gcloud ... storage cp SRC DEST` | copies one file from a bucket to disk | the same command for any `gs://` file |
| `gcloud ... cp "$ANC_SRC" lab_data/...` | the ancestry file, from the requester-pays bucket | `-billing-project` names who pays |
| `ls -lh ...; wc -l < FILE` | file sizes; lines per file | check every copy |

# Run it – Step 1: look at what arrived

When the script finishes:

```
head -n 2 lab_data/chr22_hm3.fam                        # first 2 lines: one line per person
head -n 2 lab_data/chr22_hm3.bim                        # one line per variant
head -n 1 lab_data/ancestry_preds.tsv | tr '\t' '\n'    # the header, one column name per line
```

| Piece | Meaning |
|---|---|
| `head -n N FILE` | print the first N lines of FILE (the way to look at any text file) |
| `A \| B` | a *pipe*: what A prints goes into B instead of onto the screen |
| `tr '\t' '\n'` | replace every tab with a line break, so a wide header reads as a list |

**Expected:** the script's last lines read `chr22: ... variants staged`. Everyone's count is the same.

# The files we just copied

| File (one line =) | Columns | Note |
|---|---|---|
| `chr22_hm3.fam` (person) | FID, IID, father, mother, sex, phenotype | FID is 0 for everyone. IID is the person ID. |
| `chr22_hm3.bim` (variant) | chromosome, ID, cM, position, allele 1, allele 2 | ID is an rsID; positions are GRCh38. |
| `chr22_hm3.bed` | genotypes, binary | Never open as text. |
| `ancestry_preds.tsv` (person) | `research_id`, `ancestry_pred`, `probabilities`, `pca_features`, `ancestry_pred_other` | `research_id` = CDR `person_id`. `pca_features`: 16 PCs in one text field. |

Made-up lines. One person ID, three column names: `IID`, `person_id`, `research_id`.

`chr22_hm3.fam`

```
0 1000010 0 0 2 -9
```

`chr22_hm3.bim`

```
22 rs1234567 0 17054321 A
   G
```

`ancestry_preds.tsv`

```
1000010 eur [0.01,..] [-0.009,..] eur
```

# Where phenotype data live in All of Us

| Source | What it holds | CDR table(s), and how rows are labeled |
|---|---|---|
| Program measurements | Height, weight, BMI, waist and hip, blood pressure, heart rate; taken at enrollment, some self-reported remotely | `measurement`; `measurement_source_concept_id` in the All of Us (PPI) vocabulary, e.g. 903133 |
| Surveys | The Basics, Overall Health, Lifestyle, health history, and more | `observation` (also `ds_survey`); question = `observation_source_concept_id`, answer = `value_source_concept_id` |
| Electronic health records | Diagnoses, medications, procedures, lab results and vitals, visits; from participating sites | `condition_occurrence`, `drug_exposure`, `procedure_occurrence`, `measurement`, `visit_occurrence`; standard OMOP concepts, e.g. height 3036277 |
| Wearables (Fitbit) | Daily activity, heart rate, sleep | `activity_summary`, `heart_rate_summary`, `sleep_daily_summary`, ...; one row per person-day or minute |
| Demographics | Year of birth, sex at birth, race, ethnicity | `person` |

Height is in `measurement` twice: program-measured (903133) and from health records (3036277).

# The Data Browser: finding a variable before you query

`databrowser.researchallofus.org`: public, no login; participant counts rounded to the nearest 20.

| Question | Where | What you get |
|---|---|---|
| Which survey asks about X, and how was it worded? | Surveys | the question text, answer options, participants per answer, concept IDs |
| How many participants have a diagnosis, drug, lab, or procedure? | EHR domains | concept name and ID, participant count; for labs, the value distribution |
| What does a program measurement look like? | Physical Measurements | distributions of height, weight, blood pressure, ... by sex and age |
| Is a variant in the data? | Genomic Variants | search by gene, rsID, or position; allele counts and frequencies |

The concept ID on each card goes in a query, or in a row of `data/phenotypes.tsv`. The same lookup in SQL:

```
SELECT concept_id, concept_name FROM `$CDR_DATASET.concept`
WHERE LOWER(concept_name) LIKE '%height%'
```

# Height: the concept choice

- Height is in the CDR `measurement` table: one row per measurement, often several rows per person.
- We select `measurement_source_concept_id = 903133`: height measured by the program at enrollment, plus remote self-report.
- Not the standard concept 3036277: it also returns heights from electronic health records.
- Year of birth and sex at birth come from the `person` table.

# The query and the files it writes

The first of two queries (`scripts/02_build_phenotype.sh`):

```
SELECT person_id, value_as_number          -- which columns
FROM `$CDR_DATASET.measurement`            -- which table
WHERE measurement_source_concept_id IN (903133)   -- which rows: the concept,
  AND value_as_number IS NOT NULL                 --   with a value
```

The second gets `year_of_birth` and `sex_at_birth` from `person`. Results go to `work/` (made-up values):

`work/raw_pheno.csv`

```
person_id,value_as_number
1000010,172.4
1000010,172.0
1000020,158.9
```

`work/raw_person.csv`

```
person_id,year_of_birth,sex_at_birth
1000010,1961,Female
1000020,1983,Female
```

Person 1000010 has two rows in `raw_pheno.csv`: the next step reduces them to one.

# Run it – Step 2: build the phenotype

```
DRY_RUN=1 bash scripts/02_build_phenotype.sh    # prints the two queries; runs nothing
bash scripts/02_build_phenotype.sh              # runs the queries, then cleans (about 1 min)
```

The cleaning (`scripts/build_pheno.py`, lightly simplified):

```
value = statistics.median(values)                    # 1: each person's median
if value < LO or value > HI: continue                # 2: drop outside 100-250 cm
if sex not in ('Male', 'Female'): sex = 'other'      # 3: sex at birth
age = 2026 - int(yob)                                # 4: age
z = (value - mu) / sd                                # 5: standardize within sex
```

Output `work/aou_pheno.tsv`, one row per person: `person_id value value_z age sex`.

**Expected** (the script prints the sample at each step):

```
measurement records pulled          ...
removed: outside 100-250 cm         <=20
people in analysis file             ...
```

# Exploratory data analysis: the summary table

**Input:** `work/aou_pheno.tsv` (columns `person_id value value_z age sex`). Check it from the terminal; output stays on your screen:

```
head -n 3 work/aou_pheno.tsv | column -t       # first 3 lines, columns lined up
wc -l work/aou_pheno.tsv                       # count lines: rows + 1 header line
cut -f 5 work/aou_pheno.tsv | sort | uniq -c   # keep column 5, sort it, count each value
```

**Outputs** in `results/`: `aou_pheno_summary.txt` (the table), `aou_pheno_hist.png`, `aou_pheno_kde_sex.png`.

The table has one row per group: Obs, Mean, Std. dev., p25 / Median / p75. Percentiles stand in for Min and Max, because each of those is one person's value.

Code: `scripts/pheno_eda.py - summary_row()` makes each table row

# Run it – Step 3: the summary table and figures

```
bash scripts/03_explore_phenotype.sh
cat results/aou_pheno_summary.txt
```

**Expected layout** (made-up numbers):

```
     Variable |       Obs        Mean    Std. dev.       p25      Median       p75
--------------+------------------------------------------------------------------
       height |       ...      169.10         9.80     162.0       168.9     176.2
         Male |       ...      175.90         7.40     171.0       175.9     180.8
       Female |       ...      162.40         6.90     157.8       162.3     167.0
```

**Check:** US adult means are about 176 cm (men) and 162 cm (women); SDs 7–8 cm.

Open the figures by double-clicking them in the JupyterLab file browser, folder `results/`.

# Building the PGI

For person $i$, the PGI adds up, over variants $j$, the variant's weight times the person's number of copies of the effect allele:

$$\mathrm{PGI}_i = \sum_j w_j \, g_{ij}, \qquad g_{ij} \in \{0, 1, 2\}$$

Weights $w_j$: height, Yengo et al. (2022), the GIANT GWAS of 5.3 million people; PGS Catalog score PGS002802.

| File | Contents |
|---|---|
| input `lab_data/chr22_hm3.{bed,bim,fam}` | genotypes |
| input `lab_data/height_weights.txt` | weights: `rsid effect_allele weight` |
| output `work/aou_pgi.sscore` | one row per person; `SCORE1_SUM` is the PGI |
| output `work/aou_pgi.log` | PLINK's log, including the match count |

Where a genotype is missing, PLINK uses the variant's average allele count.

# The PLINK command

From `scripts/04_build_pgi.sh` (with `CHROM=22`):

```
p2 --bfile lab_data/chr22_hm3 \
   --score lab_data/height_weights.txt 1 2 3 header cols=+scoresums list-variants \
   --out work/aou_pgi
```

| Part | Meaning |
|---|---|
| `p2` | `plink2` with our CPU and memory settings (`scripts/common.sh`) |
| `-bfile lab_data/chr22_hm3` | read `chr22_hm3.bed`, `.bim`, `.fam` |
| `-score FILE 1 2 3` | weights file; variant ID in column 1, effect allele in 2, weight in 3 |
| `header` | the first line of the weights file is column names |
| `cols=+scoresums` | also write each person's sum (`SCORE1_SUM`) |
| `-out work/aou_pgi` | prefix for the output files |

# Run it – Step 4: build the PGI

```
bash scripts/04_build_pgi.sh                              # weights, PLINK, QC histogram
grep -E 'variants processed|skipped' work/aou_pgi.log     # the match count, from the log
wc -l < lab_data/height_weights.txt                       # weight rows + 1 header line
```

**The match count.** `variants processed` = weight rows that found their variant in the genotype file. PLINK skips unmatched rows without an error.

- Close to the number of weight rows: expected.
- Near zero: the variant IDs do not match (rsIDs vs. `chr22:pos:ref:alt`), or the column numbers are wrong.

# QC on the PGI: a histogram

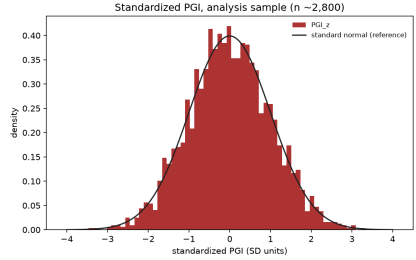
Step 4 standardizes the PGI and plots its histogram with a normal curve for reference:

$$\text{PGI_z}_i = \frac{\text{PGI}_i - \text{mean(PGI)}}{\text{SD(PGI)}}$$

```
pgi_z = (raw - raw.mean()) / raw.std()
ax.hist(pgi_z, bins=60, density=True)
```

**Expected:** a peak just below 0 and a longer right tail than the curve; skewness about $+0.6$ (`results/aou_pgi_summary.txt`).

Code: `scripts/pgi_eda.py`




Example output of this code on simulated data (not All of Us data).

Allele frequencies differ between genetic ancestries, so the real PGI is not one bell curve: one reason the regression adjusts for principal components.

# The regression

$$\text{height_z}_i = \alpha + \beta\,\text{PGI_z}_i + \gamma\,\text{age}_i + \delta\,\text{female}_i + \sum_{k=1}^{5}\theta_k\,\text{PC}_{ik} + \varepsilon_i$$

| Variable | Definition | File (step) |
|---|---|---|
| height_z | (height − mean for that sex) / SD for that sex | `work/aou_pheno.tsv` (2) |
| PGI_z | (PGI − mean) / SD, over the people in the model | `work/aou_pgi.sscore` (4) |
| age | 2026 − year of birth | `work/aou_pheno.tsv` (2) |
| female | 1 if Female, 0 if Male | `work/aou_pheno.tsv` (2) |
| $PC_1$–$PC_5$ | first 5 genetic principal components | `lab_data/ancestry_preds.tsv` (1) |

Estimated by ordinary least squares (OLS).

# What the regression reports

- $\beta$: both variables are in SD units, so $\beta$ is the difference in height, in SDs, per 1 SD of PGI, holding age, sex, and the PCs fixed.
- **The PCs** summarize broad genetic ancestry, so ancestry-related differences in height are not attributed to the PGI.
- **Incremental** $R^2 = R^2$ with $\mathrm{PGI_z} - R^2$ without it: the share of height variance the PGI explains beyond the covariates.
- **Scope**: one chromosome, and OLS, which treats participants as unrelated. A full PGI uses all 22 chromosomes.

# Run it – Step 5: the regression

```
bash scripts/05_pgi_regression.sh
cat results/aou_pgi_regression.txt
```

The fit (scripts/pgi_regression.py):

```
X[:, 0] = (X[:, 0] - X[:, 0].mean()) / X[:, 0].std()   # PGI -> PGI_z
beta, se, r2_full = fit(X)          # height_z on PGI_z, age, female, PC1-PC5
_, _, r2_cov = fit(X[:, 1:])        # the same model without PGI_z
```

**Expected layout** (made-up numbers):

```
term          estimate       SE       t
PGI_z           0.0XXX   0.00XX    X.XX
...
R2 with PGI    = 0.0XXX
R2 covariates  = 0.0XXX
incremental R2 = 0.00XX
```

# Saving results and downloading them

| Folder | Contents | Can it leave the Workbench? |
|---|---|---|
| `lab_data/` | copied-in genotypes, ancestry file, weights | No |
| `work/` | person-level files: phenotype, PGI scores | No |
| `results/` | summary tables (`.txt`), figures (`.png`) | Yes, after the check and your review |
| `runs/` | archive of the run (copied to the bucket) | No |

**To download a file:** open it and read it; then JupyterLab file browser → right-click → **Download**.

Figures and free text are not checked by the script: review them yourself. Copying out with `gcloud storage cp` is blocked; a large summary file needs an approved exception first.

# What step 6 does

`scripts/06_save_run.sh`:

1. **Checks `results/`** for counts of 1–20 (direct, or from the difference of two counts) and for person-level files; then checks the counts the steps printed. If it finds any, it stops and names the file.
2. **Writes `results/run_manifest.txt`**: the git commit ID, the settings, the PLINK version, and checksums of the input files.
3. **Packs** `config.sh`, `scripts/`, `work/`, and `results/` into one archive in `runs/`.
4. **Copies** the archive to `$WORKSHOP_BUCKET/statgen-lab/` and lists it to confirm.

The archive holds person-level files, so it stays in the workspace bucket. It is never downloaded.

Code: `scripts/check_disclosure.py; scripts/check_counts.py`

# Run it – Step 6: save, download, stop

```
bash scripts/06_save_run.sh
cat results/disclosure_report.txt                    # what the check found
grep code_version results/run_manifest.txt           # the commit that produced this run
gcloud storage ls "$WORKSHOP_BUCKET/statgen-lab/"    # list the bucket folder: the archive is
    there
```

**Expected:** `none found` twice, the printed-counts check with no pairs, then the archive's bucket path. If the check stops the save, it prints the file and the count: fix that cell and re-run.

**Download one file:** file browser → `results/` → right-click `aou_pgi_hist.png` → Download.

**Stop the VM: Apps** tab → your JupyterLab app → **Stop**. Stop keeps the app and its disk.

# All the commands, in order

```
git clone https://github.com/jeffreyswigert/aou-statgen-lab.git && cd aou-statgen-lab
cp config.example.sh config.sh && nano config.sh   # WORKSHOP_BUCKET, BILLING_PROJECT
bash scripts/00_setup.sh                # check tools and settings
bash scripts/01_fetch_genotypes.sh      # genotypes + ancestry file       -> lab_data/
bash scripts/02_build_phenotype.sh      # CDR queries, one row per person -> work/
bash scripts/03_explore_phenotype.sh    # summary table, 2 figures        -> results/
bash scripts/04_build_pgi.sh            # PGI, match count, histogram     -> work/, results/
bash scripts/05_pgi_regression.sh       # the regression                  -> results/
bash scripts/06_save_run.sh             # check, manifest, archive to the workspace bucket
```

**Settings to change** (in `config.sh`, or before one command): `PHENO=ldl`, another phenotype (`bash scripts/02_build_phenotype.sh list`); `SAMPLE_MOD=10`, 1 person in 10 (a quick trial run); `GENO_SOURCE=aou`, build the subset from All of Us's files; `CHROM=21`, another chromosome.

**After this slide:** reference slides on autostop, setting up your own workspace, building the genotype subset, and the acronyms used today.

# Thank you

Questions?

Code, README, commands, cheat sheet, slides:
`github.com/jeffreyswigert/aou-statgen-lab`

SSGAC: `https://www.thessgac.org`

Jeff Swigert: `js_850@usc.edu`

SSGAC/CESR Social-Science Genetics Methods Workshop. Generously supported by NIH/NIA Grant R24-AG065184 and the All of Us Research Program.

# Reference: autostop, when to change it

Autostop stops the VM when it has been idle. The VM counts as **idle** when, for 15 minutes,

- average CPU use is below 50%, and
- no browser tab or terminal connection is active.

**Keep autostop on** (1 hour today) for interactive work.

**Set it longer than the job** (up to 14 days) when a job runs for hours with low CPU use:

- a single-threaded program on an 8-CPU VM (1 busy CPU = 12.5% use);
- a large copy from a bucket, which waits on the network, not the CPU;
- a script that waits on a long BigQuery query.

Otherwise the VM can stop in the middle of the job. **Turn autostop off** only if you will stop the VM yourself.

# On your own: set up a workspace and add resources

Today the workspace, its data, and its bucket were set up for you. For your own project, in the Researcher Workbench:

1. **Create a workspace.** Give it an ID, a region, and the billing **pod** that pays for it. Genotypes need Controlled Tier access.
2. **Add the All of Us data.** Add the Controlled Tier v9 data collection and select **all resources**. This adds references to the CDR and the controlled bucket, not copies.
3. **Create a bucket** for your own files (a new Cloud Storage bucket resource). Its `gs://` address is `WORKSHOP_BUCKET` in `config.sh`.
4. **Find the Google project**: workspace details → **Google project** (it looks like `wb-word-word-1234`). It is `BILLING_PROJECT` in `config.sh`.
5. **Create the app** (slide 2).

On the VM, the added data appear as folders under `~/workspace/`:

```
ls ~/workspace/                                  # one folder per added resource
ls ~/workspace/vwb-aou-datasets-controlled-v9/   # shows: pooled  v7  v8  v9
```

# On your own: the genotype subset, step by step

Today step 1 copied a subset built before class from All of Us's own files. To build one yourself:

| | Step | For chromosome 22 |
|---|---|---|
| 1 | Find All of Us's genotype files for the chromosome | `.../acaf_threshold/pgen/`: `.pgen` (genotypes, 25 GB), `.pvar` (variants), `.psam` (people) |
| 2 | Get a list of the variants you want, with GRCh38 positions | HapMap3 list, `data/hm3_chr22_hg38.tsv`: 15,414 variants |
| 3 | Match the list to All of Us's variants by position and alleles | All of Us names variants `chr22:pos:ref:alt`, not by rsID; 9,944 matched |
| 4 | Keep the matched variants (and the people you want) | first `plink2` command: `--extract`, `--keep`, `--make-bed` |
| 5 | Rename the variants to rsIDs | second `plink2` command: `--update-name` |
| 6 | Check the result | count the lines of the `.bim`; look at its first rows |

The script skips sites where All of Us lists more than one alternate allele: 9,944 of 15,414 matched. The next slide runs all six steps with one command.

# On your own: build the subset from All of Us

The same step 1, told to build the subset instead of copying it:

```
GENO_SOURCE=aou bash scripts/01_fetch_genotypes.sh    # about 6 minutes for chromosome 22
```

What it does (`scripts/subset_aou_genotypes.sh`):

1. Copies All of Us's two small chromosome-22 files: variants (`.pvar`) and people (`.psam`).
2. Matches the HapMap3 list (`data/hm3_chr22_hg38.tsv`) to the variants by position and alleles.
3. Copies the 25 GB genotype file (`.pgen`) to the VM: about 5 minutes, 25 GB of free disk.
4. Runs the two PLINK commands from "Where the genotypes come from": about 1 minute.
5. Deletes the 25 GB copy. The result is the same `lab_data/chr22_hm3` files as today's.

**Other chromosomes:** add `CHROM=21` and a HapMap3 list for it (`HM3_LIST=...`; see the README).

**Autostop:** a long copy can look idle. Keep the browser tab open, or set autostop longer than the job.

# On your own: check the subset and keep it

After the build, check it before using it:

```
wc -l < lab_data/chr22_hm3.bim          # number of variants (9,944 for chromosome 22)
head -n 2 lab_data/chr22_hm3.bim        # the names should be rsIDs, not chr22:pos:ref:alt
ls -lh lab_data/chr22_hm3.*             # .bed is about 1.3 GB with everyone in it
```

Then store it in your workspace bucket, so the build runs once and not on every new VM:

```
for ext in bed bim fam; do
  gcloud storage cp lab_data/chr22_hm3.$ext "$WORKSHOP_BUCKET/genotypes/"
done
```

That is what was done for today: `scripts/prep/make_hm3_subset.sh` builds the subset and copies it to the bucket, and step 1 copies it back in seconds.

**The files stay in the Workbench.** The subset is person-level genotype data: it can go to a workspace bucket, never to your own computer or to GitHub.

# Acronyms used today: the data

| Acronym | Stands for | In this lab |
|---|---|---|
| AoU | All of Us | the research program and its data |
| CDR | Curated Data Repository | the All of Us database the queries run against (v9 today) |
| OMOP | Observational Medical Outcomes Partnership | the CDR's table layout (`measurement`, `person`, ...) |
| ETL | Extract, Transform, Load | the program's pipeline from raw records into those tables; it assigns the standard codes |
| EHR | Electronic Health Record | hospital data; LDL and HDL come only from here |
| PPI | Participant-Provided Information | All of Us's own codes; height 903133, in `measurement_source_concept_id` |
| LOINC | Logical Observation Identifiers Names and Codes | standard codes for labs and measurements; height 3036277, in `measurement_concept_id` |
| SQL | Structured Query Language | the language of the two queries in step 2 |
| CSV/TSV | Comma-/Tab-Separated Values | the plain-text tables in `work/`, `results/`, `data/` |
| VM | Virtual Machine | the cloud computer the Workbench starts for you |

# Acronyms used today: the analysis

| Acronym | Stands for | In this lab |
|---|---|---|
| GWAS | Genome-Wide Association Study | the study that estimated the per-variant weights |
| PGI, PGS | Polygenic Index, Polygenic Score | the weighted sum of genotypes from step 4; height weights: PGS Catalog PGS002802 |
| HM3 | HapMap 3 | about a million common variants; the chr22 subset keeps only these |
| ACAF | Allele Count, Allele Frequency | the All of Us genotype release filtered on those two; source of the subset |
| rsID, SNP | Reference SNP ID; Single-Nucleotide Polymorphism | a variant's name, e.g. rs12345, as used in `.bim` and the weights file |
| FID, IID | Family ID, Individual ID | the two PLINK ID columns; All of Us sets FID to 0, so the scripts join on IID |
| QC | Quality Control | plausibility bounds, the match count, the histogram |
| PC | Principal Component | genetic ancestry summaries; PC1–PC5 are regression covariates |
| OLS | Ordinary Least Squares | the regression in step 5 |
| SD | Standard Deviation | the unit of standardized height and of the PGI |